idox: what has been implemented (as of 2026-10-14)

Branch `testing2`. **Nothing described here is committed yet.** The how-to for running the new conversions is in new_conversions.md; this document is the record of what was built, why, how it was tested and what is still open.

1. At a glance

	Status
Conversions in the target matrix (7 formats, same-format excluded)	**19 of 43 exist** (9 before this work, 10 new)
Sources fully covered	**Image, PDF, Word** (every target in the matrix)
Sources not started	**Excel, XML, PPT, CSV** as inputs (24 conversions)
Model used for reading	Local **Qwen3-VL-2B** (`Q4_K_M` + `mmproj Q8_0`) on `http://127.0.0.1:8090`
Test suite	**95 tests** (19 skipped: they need Tesseract), plus an 11-case model runner
Known gaps	Searchable TIFF to PDF (no Tesseract); dense-page accuracy of the 2B model on pictures; the 10 new conversions are not in the web app or HTTP API

The matrix

✓ existed before, 🆕 built in this work, ✗ not built, – same format (excluded).

From ↓ / To →	Image	PDF	Word	Excel	XML	PPT	CSV	Done
Image	✓ (JPG to PNG/TIFF, API only)	✓	✓	✓	🆕	🆕	🆕	7 / 7
PDF	✓	–	✓	✓	🆕	✓	🆕	6 / 6
Word	🆕	✓	–	🆕	🆕	🆕	🆕	6 / 6
Excel, XML, PPT, CSV	✗	✗	✗	✗	✗	✗	✗	0 / 24

"Exists" means built and run. How well each one works is in section 6.

2. Environment set up on this laptop

The docs described a Linux machine; this is Windows 11, so the following was set up.

Item	Detail
Python	A new `.venv` on **Python 3.12** (the code uses 3.12-only syntax; 3.11 fails). `requirements.txt` installed.
llama.cpp	Build **b11247**, Windows CPU zip, in `%USERPROFILE%\.local\opt\llama.cpp\llama-b11247\` (the same build the project pins).
Model	`models_manual\Qwen3VL-2B-Instruct-Q4_K_M.gguf` (1.1 GB) and `mmproj-Qwen3VL-2B-Instruct-Q8_0.gguf` (445 MB), from `Qwen/Qwen3-VL-2B-Instruct-GGUF`.
LibreOffice	Installed with winget. Needed to draw Word files as pages.
Tesseract	**Not installed** (the winget install needs an admin approval). Blocks searchable TIFF to PDF and the OCR cross-checks.
`benchmark.py`	Ported to Windows (it used `/proc`, `pgrep`, `SIGKILL`). Hardware info, server stop/start and load average now have Windows equivalents; a missing temperature sensor no longer crashes the report. One subset run was tested; the full benchmark has not been run.

Start the model server (one copy only; two copies caused a memory surge):

```
& "$env:USERPROFILE\.local\opt\llama.cpp\llama-b11247\llama-server.exe" `
  --model models_manual\Qwen3VL-2B-Instruct-Q4_K_M.gguf `
  --mmproj models_manual\mmproj-Qwen3VL-2B-Instruct-Q8_0.gguf `
```

```
  --port 8090 --host 127.0.0.1 --no-webui --ctx-size 8192 `
  --image-min-tokens 512 --image-max-tokens 512 --parallel 1
```

3. The ten new conversions

All read the document with the 2B model except Word to Image, which only renders pages. Each has its own `<name>_input/` and `<name>_output/` folder with samples in the input folder.

Command	Conversion	How it works
`image_to_ppt.py`	Image to PowerPoint	model reads the picture, `build_pptx` writes one slide; the first short heading is the title
`image_to_xml.py`	Image to XML	model reads the picture, `to_xml.build_xml`
`image_to_csv.py`	Image to CSV	model reads the picture, `to_csv.build_csv`
`pdf_to_xml.py`	PDF to XML	`test_pdf.py --xml` (the PDF pipeline)
`pdf_to_csv.py`	PDF to CSV	`test_pdf.py --csv`
`word_to_xlsx.py`	Word to Excel	LibreOffice draws the `.docx` as a PDF, then `test_pdf.py --xlsx`
`word_to_xml.py`	Word to XML	same, `--xml`
`word_to_ppt.py`	Word to PowerPoint	same, `--pptx`
`word_to_csv.py`	Word to CSV	same, `--csv`
`word_to_image.py`	Word to PNG per page	LibreOffice draws the PDF, PyMuPDF saves each page. **No model.**

New shared code

File	Role
`to_csv.py`	CSV writer and read-back check (`build_csv`, `verify_csv`)
`to_xml.py`	XML writer and read-back check (`build_xml`, `verify_xml`)
`image_pipeline.py`	The body shared by the three Image scripts: extraction, the existing clean-ups and checks, then the writer
`doc_pipeline.py`	The body shared by the PDF and Word scripts: find LibreOffice, render Word to PDF, run `test_pdf.py`
`make_word_samples.py`	Builds the sample `.docx` files (`*.docx` is gitignored, so they are regenerated)
`test_pdf.py`	Gained `--xml` and `--csv` output flags

Output formats

- **CSV**: everything on the page in reading order. Each heading or paragraph is **one row on one line** (line breaks become spaces); each list item is one row with the bullet removed; a table is its header row then one row per data row. A blank row sits before and after every table. Rows with every cell empty are dropped (the model sometimes invents one). UTF-8 with a byte-order mark so Excel opens it. This replaced an earlier "tables only" design after the requirement changed.
- **XML**: `<document source pages><page number><heading/><paragraph/><list><item/></list><caption/><table><header><cell/></header><row><cell/></row></table></page></document>`. Always well-formed; control characters are removed.
- **PowerPoint** / **Excel**: the existing writers, unchanged.

Output behaviour worth knowing

- A picture conversion keeps **text and its structure** (headings, paragraphs, lists, tables), not the picture, colours, fonts or layout. Image to PPT puts text on the slide, not the original image.
- The Image scripts print `REVIEW NEEDED` when the model's output fails its own consistency check (check 1).

- PDF and Word runs also write `<name>.md` (the model's clean extraction) and, for Word, `<name>.pdf` (the pages the model saw), so a bad result can be traced.

4. Fix: duplicated and misordered text on dense documents

Problem

On dense pages the Word, Excel, PPT, XML, CSV and TXT outputs reported "100% of the text extracted" yet contained about **1.4x** the source's words, in the wrong order (order score 14.7 out of 100 on a 3-page PDF). Pages in these files also showed a continuation of a paragraph as a stray heading.

Root causes

1. The "is this line missing?" check compares each PDF line with the model's output as an exact line. A line the model merely **misread**, or joined into a paragraph, looked missing, and the repair step **inserted it again**. `snap_to_text_layer()` already existed to prevent exactly this and `pdf_to_txt.py` called it, but `test_pdf.py` (which feeds every other text output) never did.
2. A restored line was placed at "share of characters above it x number of blocks", which is only approximate, and large-font lines became headings.
3. Separately, the model sometimes **writes blocks in the wrong order** (a table before the caption and heading above it).

Changes

Change	Where
Call `snap_to_text_layer` on digital pages before the checks	`test_pdf.py`
New placement: align the PDF's words to the model's words in order (difflib, monotonic), treat a line as missing only if few of its words match, then **splice it into its paragraph at its exact place**, append it to the paragraph above, or add a block right after its true neighbour	`repair.py: restore_missing_lines` (used by `test_pdf.py` and `pdf_to_txt.py`)
`reorder_blocks`: locate each block in the PDF's text (a block of 6+ words by its first 6 words; a short block, such as a heading, only when its text is an **entire PDF line**) and sort back into PDF order. Conservative: a block whose location occurs more than once in the document, or that cannot be located, is not moved	`repair.py: reorder_blocks` (used by both scripts)
`drop_spurious_blocks`: remove a block of 12+ words when almost none of its words line up with the PDF at that position **and** its text is real PDF text (so a paragraph the model wrote twice goes, while text read out of a picture stays). Never touches tables or short blocks; if it removed the only copy, the restore step puts the text back	`repair.py: drop_spurious_blocks` (used by both scripts)
`IDOX_DUMP_RAW=<folder>` saves the model's raw answer per page so the repair can be replayed offline without the model	`test_pdf.py`

Measured result (replay of the model's real saved output on two 3-page documents)

Document	Variant	Order (100 = perfect)	Size vs source	Blocks
Sample PDF	before	14.7	1.38x	87
	+ snap	80.7	1.10x	54
	+ new placement	89.2	1.06x	27
	+ drop repeated blocks (shipped)	**91.2**	**1.01x**	26
Word-derived	before	33.1	1.44x	96
	+ snap	72.1	1.10x	60

	+ new placement (and drop repeated blocks: no change)	**90.6**	**1.05x**	28

Recall stayed 100% and precision rose from about 84% to 96%. On `sample_test_document.pdf` a real run hit the page-2 misorder, `reorder_blocks` corrected it, and the order score went from 93 to **99**. Replay script: `sample_local_repair_test/replay.py`.

An experiment that was rejected

`restore_missing_lines(scan_all=True)` restores every PDF line the alignment cannot find, not only the ones `check_coverage()` flags. It raised the order score on the sample PDF (91.2 to 94.3) but dropped it on the Word-derived document (90.6 to **48.1**): where the same sentences repeat many times (Lorem ipsum does), the alignment cannot tell which copy a line belongs to and it put restored text in the wrong place. A dropped line that does not repeat is already flagged by the existing check, so the only extra lines `scan_all` finds are repeated ones, which cannot be placed reliably. It stays in the code as an opt-in parameter, off in the pipeline. `sample_local_repair_test/split.py` reproduces the comparison.

Limits

- A dropped line that repeats text elsewhere on the page is never flagged (see the experiment above: it cannot be fixed reliably, only documented).
- A single-word block, or a short block that is not a whole PDF line, cannot be located and stays behind the block before it.
- Order is about 90 to 91, not 100. What is left is misread words the snap step could not correct (`seclerem`, `suspensidse`), which is the model's own error, and text on highly repetitive pages where alignment is ambiguous.

5. Other fixes

File	Fix
`word_to_pdf.py`, `image_to_pdf.py`	LibreOffice was given `file://C:\...`, which is not a valid address on Windows, so conversion hung until it timed out. Now `Path.as_uri()`. `image_to_pdf.py` also uses absolute paths and finds LibreOffice in `C:\Program Files`.
`image_pipeline.py` / `doc_pipeline.py`	A missing input file now counts as a failure (it used to exit 0 with nothing done).
`to_csv.py`	Drops fully empty rows (found when the model invented one on `fol.png`).

6. Testing and results

Test tooling

Tool	What it does
`test_new_conversions.py`	Runs the 10 conversions (11 cases) on the local model. Deletes old output, runs the script, **re-opens the written file**, checks shape and required words. `--list`, `--only`, `--all-samples`, `--verbose`.
`tests/test_new_writers.py`	20 model-free tests of the CSV and XML writers and that every new script imports and has its folders
`tests/test_repair.py`	16 model-free tests: 5 for `restore_missing_lines`, 7 for `reorder_blocks` (including the real page-2 order and short headings), 4 for `drop_spurious_blocks`
`sample_local_*_test/verify_outputs.py`	Score outputs against the source's own text: recall, precision, order, size

Full suite: `python -m unittest discover -s tests` gives **95 tests, OK** (59 existing, 20 writer, 16 repair).

Test campaigns

Campaign	Input	Result
New conversions on codebase samples	one sample each	**11 of 11 pass**, re-run after the repair change: 307 s
Original PDF to Word/Excel/PPT, plus XML, CSV, TXT	`sample_test_document.pdf`	100% words, no extras, order 99, size 1.02x; TXT verdict PASS
PDF to every conversion	a 3-page digital PDF of dense text	JPG, JPEG, PNG, TIFF exact (3 of 3 pages verified); text formats had the duplication bug (fixed, section 4)
Image to anything	the 3 page pictures	PDF, PPT, XML, CSV: **87%** of words (89 / 85 / 87), order 82; Word: **77%**; Excel refused all 3 (no table, correct); JPG to PNG/TIFF fine
Word to anything	a 3-page `.docx`	PDF, Image fine; Excel/XML/PPT/CSV had the duplication bug (fixed)

Where the model is used

The 2B model reads each page picture (`test_pdf.py` calls `extract_page()`, which posts to `/v1/chat/completions`). LibreOffice, the checks and the writers are plain code. With the model unreachable a Word to Excel run fails and writes no `.xlsx`, which confirms the content comes from the model's reading. On digital pages the pipeline also **copies missing lines back from the PDF's own text layer**, so some final text is restored rather than read by the model.

7. Open items and known limitations

1. **Dense pictures:** with no text layer to repair from, Image conversions show the raw 2B model: it reads about 77 to 87% of a dense page. Options not yet tried: a higher image-token setting, or splitting a page into halves before reading.
2. **Searchable TIFF to PDF** and the OCR cross-checks need Tesseract (`winget install UB-Mannheim.TesseractOCR`, admin).
3. **The model is not deterministic:** the same image can give different results on different runs, so check the output against the source, not only the PASS line.
4. **Repeated dropped lines** are not detected and cannot be restored reliably (section 4, rejected experiment).
5. **PowerPoint** can warn that a slide or table is overcrowded and compress it. This follows a deliberate rule in `to_pptx.py`: one PDF page is always one slide and is never split. Changing it (for example, continuation slides) is a design decision, not a bug fix.
6. **Not wired in:** the 10 new conversions are command-line only; they are not routes in `api/idox_api.py` or `idox_app.py`.
7. **Remaining matrix:** Excel, XML, PPT and CSV as inputs (24 conversions). Open design question: read them through the model (render to pages, then read) or parse the structured ones directly in code.
8. **Not committed.** About 50 files are changed or new. The `sample_local_*_test/` folders hold test output and logs and should probably be left out of a commit.

8. File index

New	Modified
`to_csv.py`, `to_xml.py`, `repair.py`	`test_pdf.py` (flags, snap, new repair, reorder, raw dump)
`image_pipeline.py`, `doc_pipeline.py`	`pdf_to_txt.py` (new repair, reorder)
`image_to_{ppt,xml,csv}.py`, `pdf_to_{xml,csv}.py`, `word_to_{xlsx,xml,ppt,csv,image}.py`	`word_to_pdf.py`, `image_to_pdf.py` (Windows fixes)
`<each>_input/`, `<each>_output/` for the ten	`benchmark.py` (Windows port)

`make_word_samples.py`, `test_new_conversions.py`	
`tests/test_new_writers.py`, `tests/test_repair.py`	
`docs/new_conversions.md`, `docs/implementation_summary.md`	
`sample_local_pdf_test/`, `sample_local_other_test/`, `sample_local_3page_test/`, `sample_local_repair_test/` (test inputs, outputs, logs, checkers, `replay.py`)	